

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UM Hackathon 2026

The Bare Minimums

Domain: AI for Economic Empowerment & Decision Intelligence

Version 1.0 | April 2026

Table of Contents

1	Introduction	4
1.1	Purpose.....	4
1.1.1	Architecture	4
1.1.2	Data Flows.....	5
1.1.3	Model Process	6
1.1.4	Role of Reference.....	7
1.2	Background.....	7
1.3	Previous Version / Baseline.....	8
1.4	Target Stakeholders.....	9
2	System Architecture & Design	10
2.1	High-Level Architecture Overview.....	10
2.2	LLM as Service Layer — Google Gemma 4 26B Google DeepMind Integration	12
2.2.1	Dependency Diagram Overview	12
2.2.2	Sequence Diagram — AI Daily Briefing Generation (Core Feature)	15
2.3	Technological Stack	16
3	Key Data Flows.....	19
3.1	Data Flow Diagram (DFD) Overview	19
3.2	Normalized Database Schema (3NF).....	20
3.2.1	Module 1 — Identity & Access	20
3.2.2	Module 2 — Inventory	20
3.2.3	Module 3 — Financial	22
3.2.4	Module 4 — Staff	23
3.2.5	Module 5 — AI & Briefing	24
4	Functional Requirements & Scope	27

4.1	Minimum Viable Product (MVP)	27
4.2	Non-Functional Requirements (NFRs)	27
4.3	Out of Scope / Future Enhancements	28
5	Monitor, Evaluation, Assumptions & Dependencies	30
5.1	Technical Evaluation.....	30
5.1.1	Grayscale Rollout & A/B Testing.....	30
5.1.2	Emergency Rollback (ER) & Golden Release	30
5.2	Priority Matrix.....	30
5.3	Monitoring Plan.....	31
5.4	Assumptions	32
5.5	External Dependencies.....	32
6	Project Management & Team Contributions	35
6.1	Project Timeline	35
6.2	Team Member Roles.....	36
6.3	Recommendations	36

1 Introduction

This document introduces the strategic and technical solution developed in response to the problem statement identified during the product review phase. The platform, an AI-powered business management system for SME F&B operators, consolidates fragmented operational data into a single intelligent platform, enabling proactive decision-making through the Google Gemma 4 26B model.

1.1 Purpose

This System Analysis Document (SAD) defines the technical scope and design decisions behind the SME F&B AI Business Management Platform. The key system elements covered are:

1.1.1 Architecture

The platform is a three-layer cloud-hosted system built initially for F&B businesses and designed to expand across SME verticals. The **Data Layer** aggregates business inputs from all entry points, like POS system syncs (inventory consumption, sales transactions), manual entries, and file uploads such as CSVs and scanned receipts, normalising everything into a Supabase PostgreSQL store with row-level security.

The **AI Layer (Google Gemma 4 26B)** runs continuously against this data to execute four core business logic flows: syncing POS data to trigger restocking alerts when inventory crosses thresholds, scanning financial transactions to flag anomalies against historical patterns, analysing staff schedules against sales volume to surface labour cost optimisation opportunities, and generating a consolidated daily briefing each morning that summarises the prior day's performance and risks.

The **Dashboard Layer** renders all of these as actionable insights for the SME owner, for instance, restock prompts, anomaly warnings, overall business health, and the morning briefing pop up card, with every owner acknowledgement fed back into the system to continuously improve future recommendations.

1.1.2 Data Flows

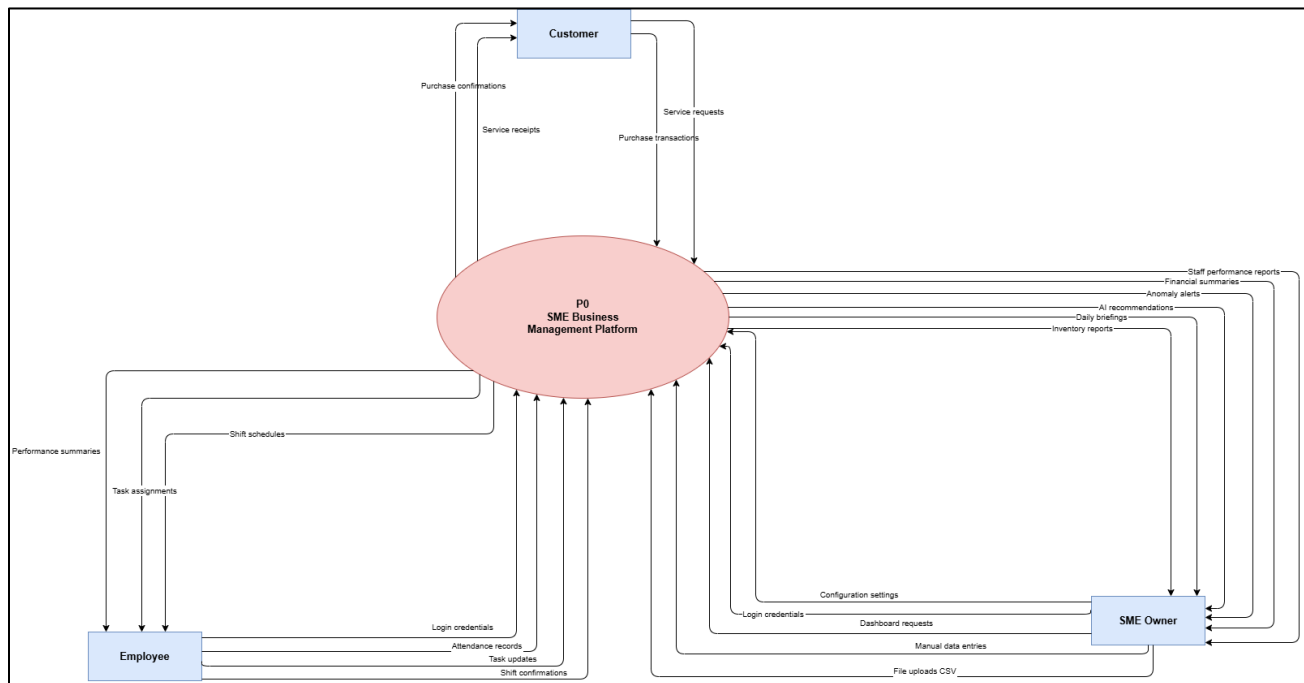


Figure 1: Context Diagram

The SME Business Management Platform interfaces with three external entities across its system boundary. The **Customer** submits purchase transactions and service requests into the platform and receives purchase confirmations and service receipts in return. The **SME Owner** is the primary human actor, supplying login credentials, dashboard requests, manual data entries, file uploads in CSV format, and configuration settings into the system, and receiving daily briefings, AI recommendations, anomaly alerts, financial summaries, inventory reports, and staff performance reports as outputs. The **Employee** submits login credentials, attendance records, task updates, and shift confirmations into the platform and receives shift schedules, task assignments, and performance summaries in return.

1.1.3 Model Process

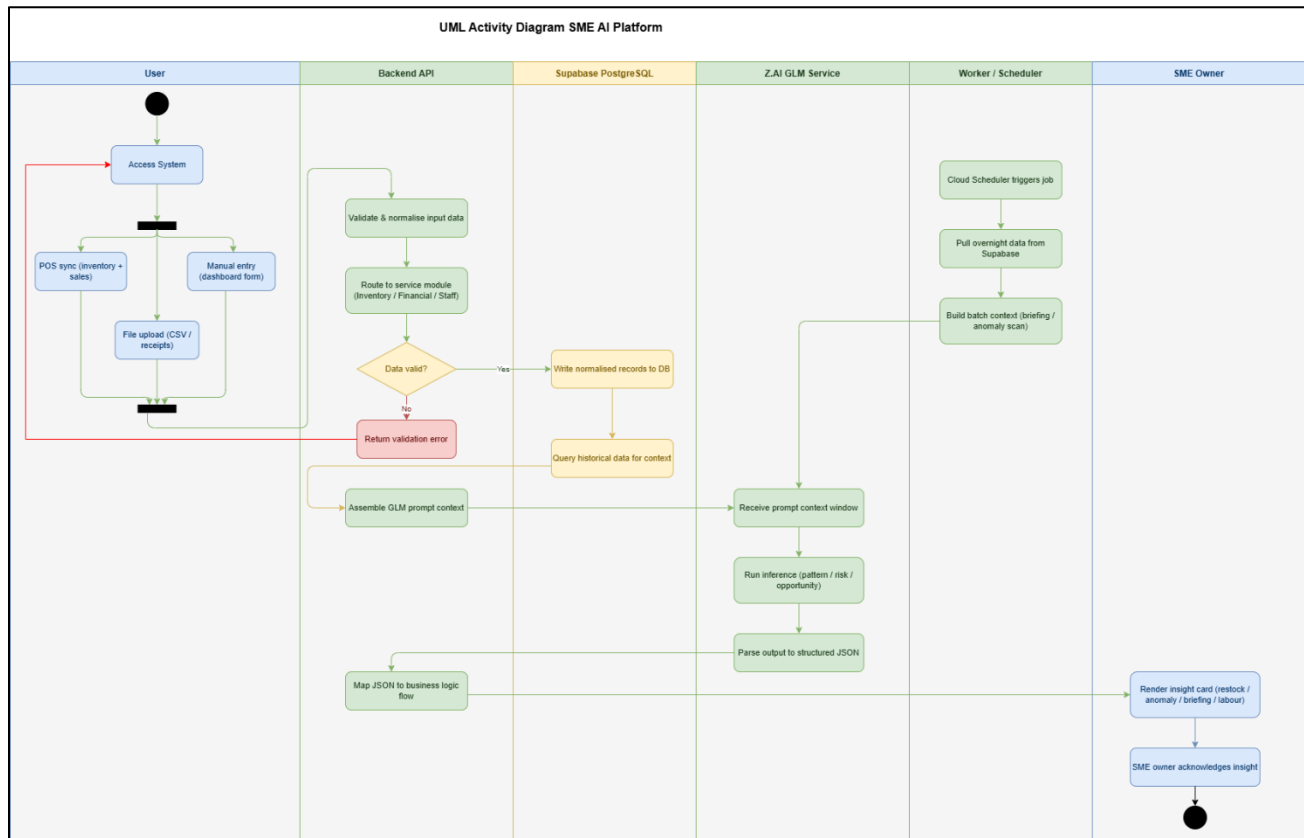


Figure 2: System Activity Diagram

This activity diagram shows the end-to-end workflow of your SME AI platform, organized using swimlanes to represent different system components (User, Backend API, Database, AI service, Scheduler, and SME Owner). The process starts when the **user** accesses the system and provides data through POS integration, manual input, or file uploads. This data flows into the **Backend API**, where it is validated and normalized before being routed to the appropriate service module (inventory, financial, or staff). If the data fails validation, an error is returned; otherwise, it is stored in the database and enriched with historical data for context.

From there, the system either triggers automatically via the scheduler or through real-time API calls. The **Worker/Scheduler** prepares the data and sends it to the **AI (Google Gemma 4 26B)** service, where inference is performed to detect patterns, risks, or opportunities. The AI output is structured into JSON and sent back to the **Backend API**, which maps it into specific business logic flows such as restocking alerts, anomaly detection, labour recommendations, or daily briefings. Finally, these

insights are presented on the SME owner dashboard, where the user can review and acknowledge them, completing the process.

1.1.4 Role of Reference

This document serves as the primary technical reference for all developers, QA testers, and evaluators to understand the system's high-level architecture, data design, and functional boundaries throughout UMHackathon 2026.

- **Technical Onboarding:** Provides the Development Team with clear API contracts and a modular codebase overview and assists the QA Team in defining test scopes for AI recommendation accuracy and data flow integrity.
- **Design Decision Baseline:** Defines the technical scope and design decisions behind the platform, including the transition from manual, fragmented processes to a unified AI-driven layer.
- **Operational Blueprint:** Outlines the end-to-end workflow, from data ingestion via POS or manual entry to the generation of actionable insights by the Google Gemma 4 26B, ensuring all stakeholders understand the system's "no-black-hole" data policy.
- **Scalability Roadmap:** Establishes the architectural foundation required to move beyond the initial F&B implementation into other SME sectors such as retail and light manufacturing.

1.2 Background

Small and medium-sized enterprises (SMEs) in the food and beverage (F&B) sector, particularly restaurants and cafés, commonly operate using fragmented and disconnected systems. These typically include standalone point-of-sale (POS) systems, manual spreadsheets for financial and inventory tracking, and informal methods for staff scheduling. As a result of this lack of integration, critical business data becomes siloed across multiple platforms, leading to inefficiencies such as delayed decision-making, inconsistent record-keeping, and higher error rates in daily operations. SME owners are therefore often constrained to a reactive management approach, where issues such as stock shortages, cash flow imbalances, and staffing gaps are only addressed after they have already impacted operations.

To address these challenges, the proposed platform introduces a unified, AI-driven layer that consolidates data from existing operational tools without requiring SMEs to replace their current systems. By aggregating and interpreting data from POS systems, financial records, and workforce inputs, the platform transforms raw operational data into actionable insights. This enables SME owners to move from reactive decision-making to a more proactive and data-informed management approach, improving overall efficiency and operational visibility.

1.3 Previous Version / Baseline

There is no existing version of the proposed platform; instead, the baseline is based on common operational realities among many SMEs in the F&B sector. In this baseline state, businesses often rely on manual and disconnected processes, including spreadsheet-based inventory and financial tracking, paper-based or verbally communicated staff scheduling, and standalone POS systems that do not fully integrate with other business tools or provide advanced analytics. In addition, decisions such as equipment maintenance, subscription renewals, and restocking are often handled reactively or on an ad hoc basis rather than through structured planning.

This platform is designed to sit above and integrate with these existing practices rather than replace them entirely. It introduces an intelligent, AI-augmented layer that consolidates disparate data sources and adds real-time analysis and predictive insights, helping SMEs move toward a more automated, efficient, and proactive operating model.

1.4 Target Stakeholders

Stakeholder	Role	Expectations
Customer	Browse menus, place orders, track delivery via dashboard/portal	Simple AI-recommended ordering experience; accurate demand insights
Restaurant/Café	Receive inventory alerts, seasonal menu suggestions, equipment maintenance reminders	Timely notifications, correct AI recommendations, minimal manual input
SME Owner	Monitor financial health, manage staff payroll, review daily AI briefings	Centralized view, proactive alerts, human control over all final decisions
Development Team	Build and maintain platform modules (Financial, Inventory, Staff, AI layer)	Clear API contracts, modular codebase, well-documented architecture
QA Team	Validate AI recommendation accuracy, data flow integrity, and dashboard behavior across all user roles	Defined test scope, request/response formats, edge case coverage

2 System Architecture & Design

2.1 High-Level Architecture Overview

Architecture

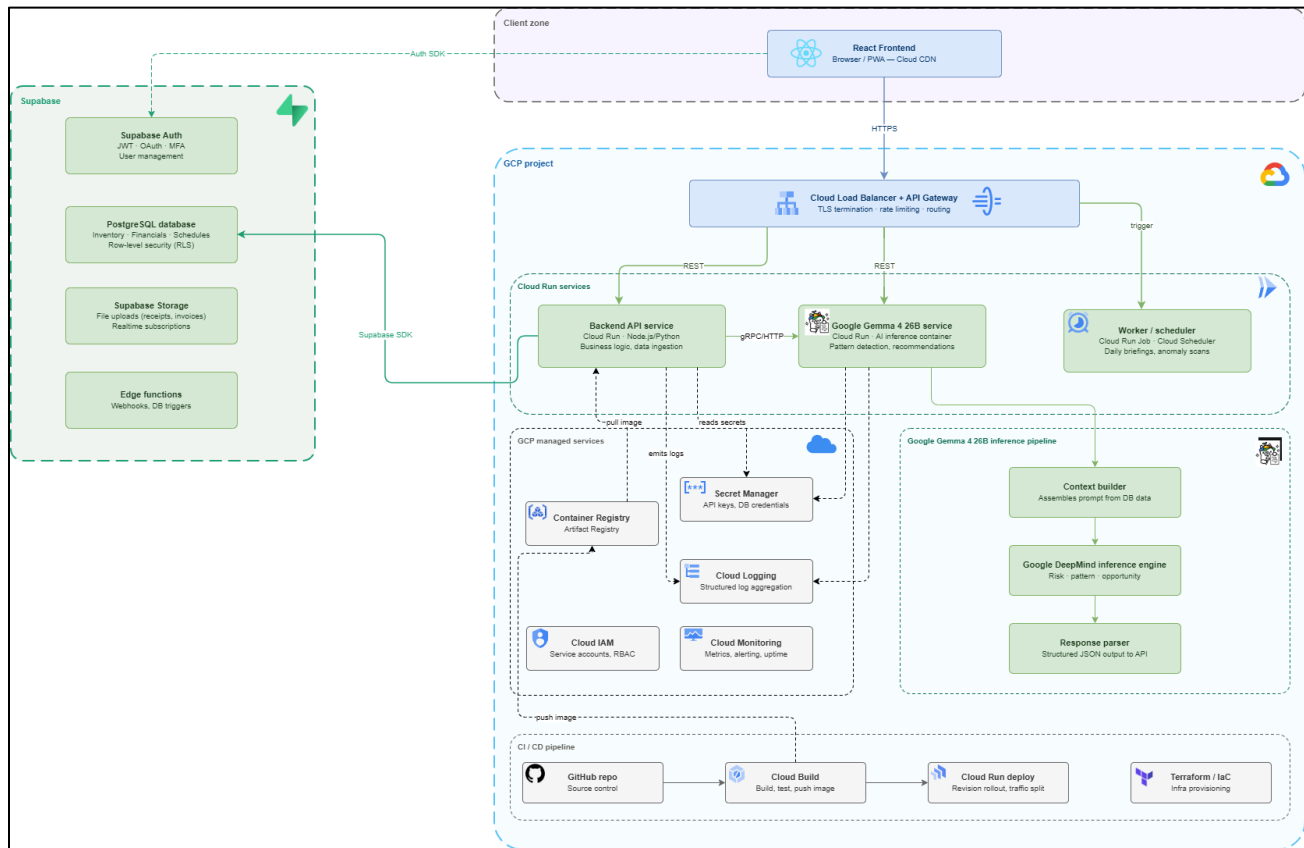


Figure 3: System Architecture

The platform is a three-zone cloud architecture hosted on GCP, built initially for F&B businesses and designed to scale across SME verticals. Users interact through a **React frontend** served via Cloud CDN, which routes all requests through a **Cloud Load Balancer** and **API Gateway** handling TLS termination and rate limiting. Behind the gateway, **three Cloud Run containers** do the heavy lifting. The **Backend API service** handles business logic and labour cost analysis, and daily AI briefing generation. In contrast, the **Google Gemma 4 26B Service** runs the AI inference pipeline, with context building to pattern analysis and final structured JSON output, and a **Worker/Scheduler** handles background jobs like daily briefings and anomaly scans. Business data enters the system from all directions: POS integrations, manual dashboard entries, and file uploads such as CSVs and scanned

receipts. All containers pull their images from Artifact Registry, read credentials from Secret Manager, and emit structured logs to Cloud Logging and Monitoring.

The data layer is split between GCP and an external Platform as a Service (PaaS). **Supabase** provides the PostgreSQL database with row-level security, JWT-based authentication, file storage, and edge function triggers. Here, the Backend API communicates with it via the Supabase SDK, while the React frontend connects directly for authentication. The entire infrastructure is defined as code using **Terraform**, built and tested by **Cloud Build**, and deployed as *Cloud Run* revisions, in order to provide a clean *GitOps* pipeline from *GitHub* commit to live service. Finally, **Cloud IAM** governs service account permissions across every component, ensuring least-privilege access throughout.

Architecture Breakdown

Component	Details
System	Web-based business management dashboard (browser + optional PWA)
Architecture	3-layer: Data Layer → AI Layer (Google Gemma 4 26B) → Dashboard Layer
Deployment	GCP cloud-hosted; modular Cloud Run microservices
AI engine	Google Gemma 4 26B — prompt inference, pattern recognition, structured JSON output
Data sources	POS system integrations, manual dashboard entries, file uploads (CSV, receipts)
Database	Supabase PostgreSQL with row-level security (RLS) and JWT-based auth
Storage	Supabase Storage for file uploads; Supabase Edge Functions for DB triggers
Caching	Supabase in-built connection pooling; Google DeepMind context caching via Backend API

Infrastructure	Terraform (IaC), Cloud Build CI/CD, Artifact Registry, Cloud Run revisions
Observability	Cloud Logging, Cloud Monitoring, Secret Manager, Cloud IAM (least-privilege)

The platform is structured as a client-server system with a web-based dashboard as the primary client interface. SME owners access the system via a browser, with an optional PWA for mobile use. The Backend API layer handles all data processing, AI orchestration, and persistence, deployed as a Cloud Run container on GCP. Three logical service modules, Inventory Service, Financial Service, and Staff Service, operate independently and communicate through the Cloud Load Balancer and API Gateway. All services share a PostgreSQL database hosted on Supabase, with connection pooling and Supabase Edge Functions handling database-level triggers and webhooks. The Google Gemma 4 26B service runs as a dedicated Cloud Run container, receiving structured context from the Backend API and returning pattern analysis and recommendations as JSON. Background jobs such as daily briefings and overnight anomaly scans are handled by a Cloud Run Worker triggered via Cloud Scheduler.

2.2 LLM as Service Layer — Google Gemma 4 26B Google DeepMind Integration

2.2.1 Dependency Diagram Overview

The following describes how Google Gemma 4 26B Google DeepMind is integrated as a dedicated service layer within the platform architecture:

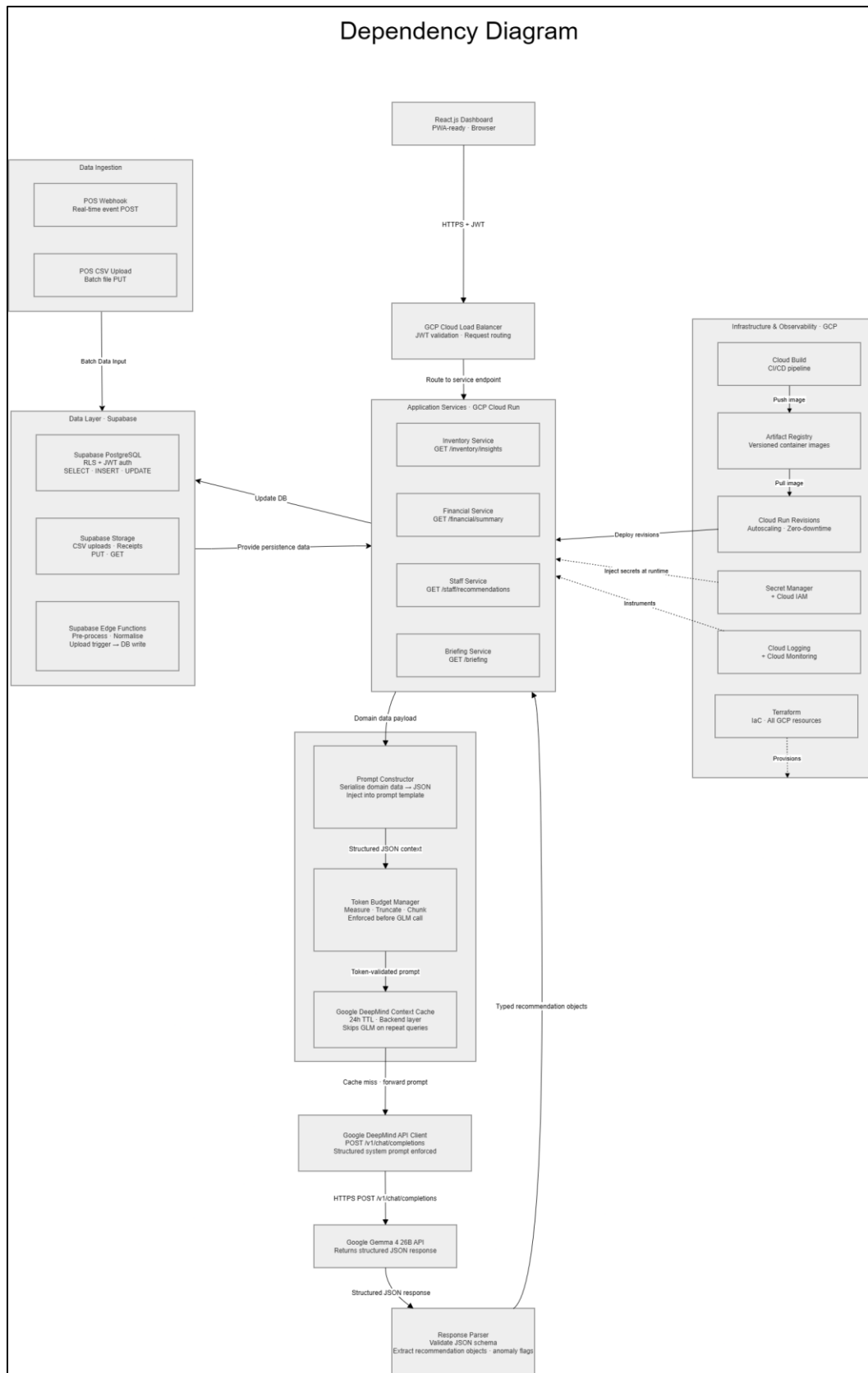


Figure 4: Dependency Diagram

The platform is structured across three layers, Dashboard Layer, AI Layer, and Data Layer — each deployed as independent Cloud Run services on **GCP**, provisioned via Terraform, and delivered through Cloud Build CI/CD pipelines. The React.js frontend (PWA-ready) communicates through GCP Cloud Load Balancer, which handles JWT validation and routes each inbound request to the correct Cloud Run microservice: Inventory Service, Financial Service, Staff Service, or Briefing Service. Every service is independently versioned via Cloud Run revisions, allowing zero-downtime deployments and per-service autoscaling without cross-service coupling.

Prompt construction begins at the module level. Each Cloud Run service (Inventory, Financial, Staff, Briefing) owns a Prompt Constructor that serialises its domain data, like inventory transactions, revenue records, and headcount logs, into structured JSON and injects it into a fixed-length prompt template. This serialised context is then passed to the **Token Budget Manager**, which enforces token limits before anything reaches **Google DeepMind**: inputs are measured, truncated to the configured budget ceiling, and chunked if the payload exceeds a single context window. This gate is the last preprocessing step before the Google DeepMind API Client dispatches the request. No raw or unvalidated data is ever sent directly to the Google DeepMind API.

The Google DeepMind API Client sends an HTTPS POST to */v1/chat/completions* on the Google Gemma 4 26B API. The system prompt within each request enforces structured JSON output as Google DeepMind is not called in open-ended mode. Responses flow back to the Response Parser, which validates the returned payload against an expected schema and extracts typed recommendation objects and anomaly flags before passing them back to the calling service. A Google DeepMind Context Cache sits at the backend layer: repeated or near-identical queries within a 24-hour window are served from cache, avoiding redundant token spend and reducing Google DeepMind API latency for the end user.

All persistence is handled through **Supabase**. Each Cloud Run service communicates with Supabase PostgreSQL via the Supabase client SDK, with row-level security (RLS) enforced at the database layer so that each query is automatically scoped to the authenticated tenant. File-based inputs

(CSV exports from POS systems, scanned receipts) are written to Supabase Storage, which triggers Supabase Edge Functions to pre-process and normalise the files before the resulting records are committed to PostgreSQL. POS integrations also push data directly into the Inventory and Financial services via webhook, providing a real-time ingestion path alongside the file upload route.

Observability and security are wired across all distributed components. Cloud Logging and Cloud Monitoring instrument every Cloud Run service. Secret Manager holds all credentials and API keys, injected at runtime via Cloud IAM bindings under a least-privilege model. No service has broader IAM permissions than its operational scope requires. Artifact Registry stores all container images produced by Cloud Build, and Cloud Run revisions pull exclusively from this registry.

2.2.2 Sequence Diagram — AI Daily Briefing Generation (Core Feature)

Step-by-step walkthrough of the Daily Briefing feature from SME Owner interaction to Google DeepMind response

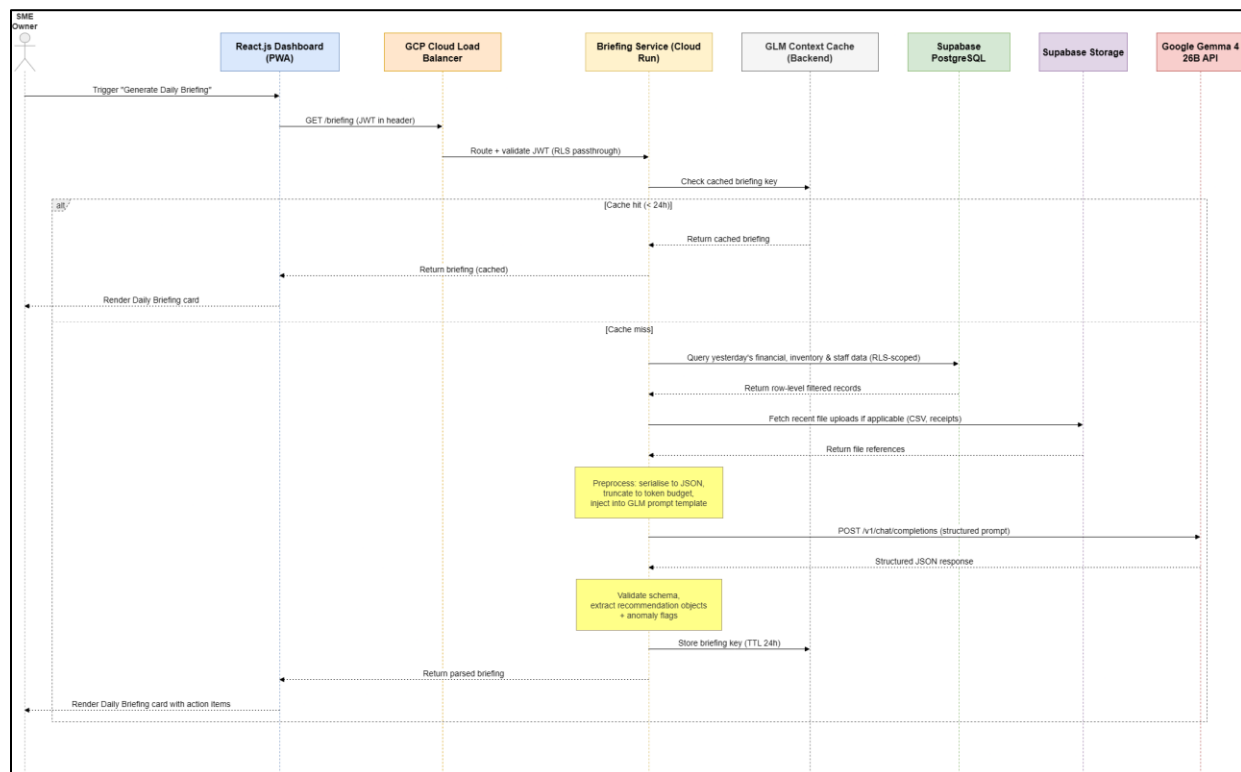


Figure 5: Sequence Diagram

The Daily Briefing generation flow follows the same steps as before, updated to reflect the GCP and Supabase stack. The SME Owner triggers the briefing manually or on schedule from the React.js dashboard. The request travels through GCP Cloud Load Balancer, carrying a JWT for authentication to the Briefing Service running on Cloud Run. The service first checks the Google DeepMind Context Cache maintained at the backend layer; a hit within the 24-hour TTL window returns the briefing immediately without touching the database or Google DeepMind.

On a cache miss, the Briefing Service queries Supabase PostgreSQL for the prior day's financial, inventory, and staff records. Row-level security ensures the query is scoped strictly to the authenticated tenant. If any file-based data — such as a CSV upload or receipt scan — is relevant to the briefing window, references are fetched from Supabase Storage. The combined dataset is then preprocessed: serialised to JSON, truncated to fit the token budget, and injected into the Google DeepMind prompt template. The Briefing Service dispatches the structured prompt to Google Gemma 4 26B via HTTPS POST. The response, returned as structured JSON, is validated against the expected schema and parsed to extract prioritised action items and anomaly flags. The result is stored in the backend cache with a 24-hour TTL before being returned to the frontend, where the React.js dashboard renders the Daily Briefing card with actionable items highlighted.

2.3 Technological Stack

Layer	Technology	Justification
Frontend	React.js	Component-based UI architecture suited for a modular, data-dense dashboard; accessible to non-technical SME owners with minimal learning curve
Backend	FastAPI (Python)	Lightweight, async-capable RESTful API layer; native Python ecosystem simplifies Google

		DeepMind prompt construction, JSON serialisation, and response parsing with minimal boilerplate
Database	Supabase PostgreSQL	Managed relational database for financial records, inventory transactions, staff data, and audit logs; row-level security (RLS) and JWT-based auth enforce tenant data isolation at the database layer
AI / LLM	Google Gemma 4 26B API	Core reasoning engine for pattern analysis, anomaly detection, restocking recommendations, and daily briefing generation; called via structured HTTPS POST with enforced JSON output
Storage	Supabase Storage	- Stores static frontend assets, CSV uploads, and receipt files - Supabase Edge Functions trigger on upload to pre-process and normalise file data before committing to PostgreSQL
Caching	Google DeepMind Context Cache (Backend API layer)	In-memory cache maintained at the FastAPI service level for Google DeepMind query results; repeated briefing or recommendation requests within a 24-hour TTL window are served from cache, reducing Google DeepMind API token spend and latency
Infrastructure as Code	Terraform	Provisions and manages all GCP resources declaratively ensures reproducible environments across development, staging, and production
CI/CD	Cloud Build + GitHub Actions	- Cloud Build handles container build and push to Artifact Registry on merge

		- GitHub Actions runs independent pipelines per service module, isolating deployments so a change to the Inventory Service does not trigger a full-stack redeploy
Observability	Cloud Logging, Cloud Monitoring, Secret Manager, Cloud IAM	- Cloud Logging and Monitoring instrument all Cloud Run services - Secret Manager injects credentials at runtime - Cloud IAM enforces least-privilege access across all deployed components

3 Key Data Flows

3.1 Data Flow Diagram (DFD) Overview

Data flows through the system across four key pathways:

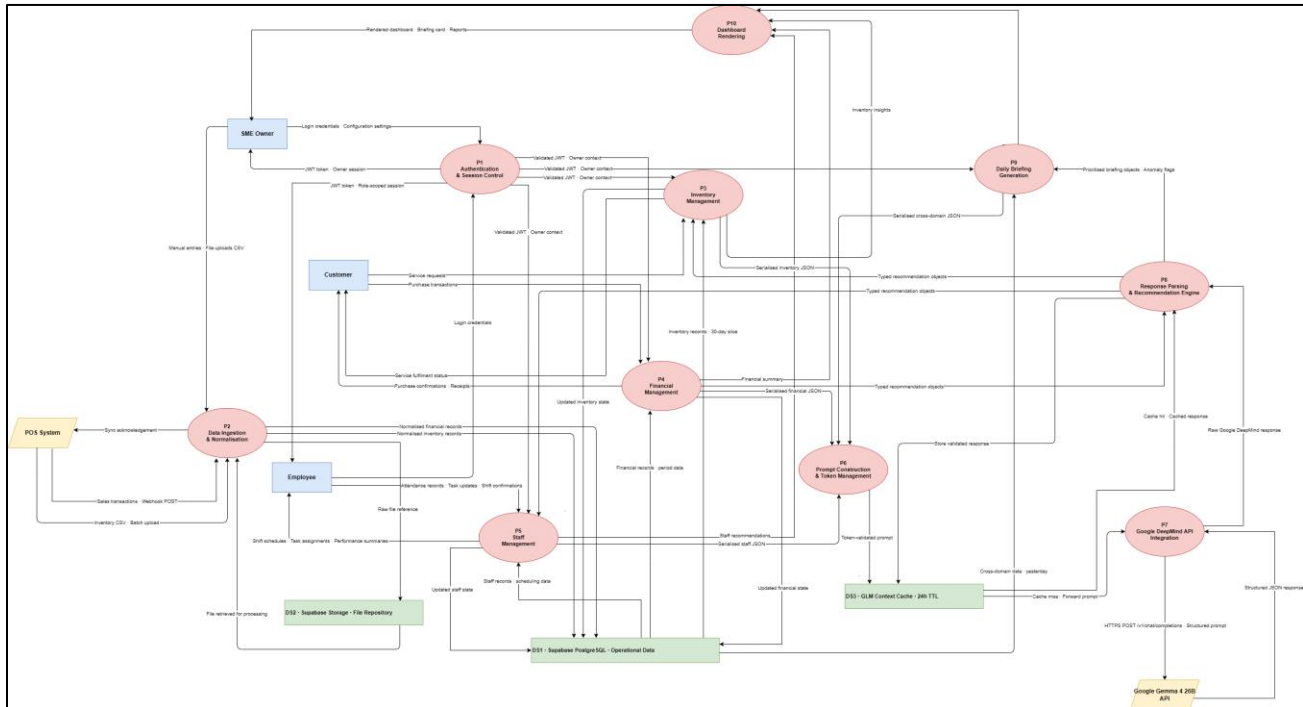


Figure 6: DFD Diagram

- Inventory Flow: POS export / manual upload → Data Ingestion API → Normalization Layer → PostgreSQL (inventory_items, stock_transactions) → Google DeepMind Inventory Analyzer → Restocking Recommendations → Dashboard
- Financial Flow: Manual entry / POS financial export → Financial Service API → PostgreSQL (expenses, salaries, subscriptions) → Google DeepMind Financial Analyzer → Anomaly flags, burn rate projection → Dashboard
- Staff Flow: Schedule upload / manual input → Staff Service API → PostgreSQL (staff_records, shifts, payroll) → Google DeepMind Staff Optimizer → Workforce allocation recommendations → Dashboard
- Briefing Flow: Aggregated daily snapshot (all modules) → Briefing Service → Redis cache check → Google DeepMind Briefing Generator → Parsed JSON → Dashboard Daily Briefing card

All data flows enforce the no-black-hole rule: every external input produces an observable output (stored record, alert, or recommendation). No data enters the system without being persisted and acknowledged.

3.2 Normalized Database Schema (3NF)

3.2.1 Module 1 — Identity & Access

users — master tenant record; all other tables scope to this via `user_id` FK

Column	Type	Constraint	Notes
<code>user_id</code>	UUID	PK, DEFAULT <code>gen_random_uuid()</code>	Supabase-generated UUID
<code>name</code>	VARCHAR(100)	NOT NULL	Business owner full name
<code>email</code>	VARCHAR(255)	NOT NULL, UNIQUE	Login identity
<code>business_type</code>	VARCHAR(50)	NOT NULL	e.g. F&B, Retail, Services
<code>role</code>	VARCHAR(20)	NOT NULL, DEFAULT 'owner'	owner / employee / admin
<code>created_at</code>	TIMESTAMPTZ	NOT NULL, DEFAULT <code>now()</code>	Tenant registration timestamp

3.2.2 Module 2 — Inventory

inventory_items — master catalogue of trackable stock items per business

Column	Type	Constraint	Notes
<code>item_id</code>	UUID	PK, DEFAULT <code>gen_random_uuid()</code>	Surrogate primary key
<code>user_id</code>	UUID	FK → <code>users(user_id)</code> , NOT NULL	Tenant scoping
<code>name</code>	VARCHAR(150)	NOT NULL	item or ingredient name

category	VARCHAR(80)	NOT NULL	Meat Seafood Vegetables Dairy Dry Goods etc.
quantity	DECIMAL(10,2)	Default 0	Current stock quantity
unit	VARCHAR(20)	NOT NULL	kg g L ml pcs box bag bottle
reorder_level	DECIMAL(10,2)	DEFAULT 0	Quantity threshold that triggers low-stock alert
cost_per_unit	DECIMAL(10,2),	DEFAULT 0	Purchase cost per unit in business currency
supplier	VARCHAR(150)	Default NULL	Supplier name for reorder reference
expiry_date	DATE	Default NULL	Best-before or expiry date for perishables
status	VARCHAR(50)	Default 'ok'	ok low critical expired — computed on fetch
created_at	TIMESTAMPTZ	Default now()	Row creation timestamp
updated_at	TIMESTAMPTZ	Default now()	Auto-updated via trigger

inventory_logs — Immutable audit log of every stock quantity change

Column	Type	Constraint	Notes
logs_id	UUID	PK, DEFAULT gen_random_uuid()	—
item_id	UUID	FK → inventory_items(item_id), NULL	Item referenced
user_id	UUID	FK → users(user_id), NOT NULL	Tenant scoping for RLS
change_type	TEXT	NOT NULL	add / deduct / adjust / invoice
quantity_change	DECIMAL(10,2)	NOT NULL	positive = added, negative = deducted

notes	TEXT	Default NULL	Optional context e.g. "Invoice #INV-0042"
transaction_date	TIMESTAMPTZ	Default now()	Signed delta — positive = added, negative = deducted
notes	TEXT	Default NULL	Optional context e.g. "Invoice #INV-0042"
created_at	TIMESTAMPTZ	Now()	pos_sync / manual / csv_upload

3.2.3 Module 3 — Financial

expenses — records all outgoing financial transactions per business

Column	Type	Constraint	Notes
expense_id	UUID	PK, DEFAULT gen_random_uuid()	—
user_id	UUID	FK → users(user_id), NOT NULL	Tenant scoping
category	VARCHAR(80)	NOT NULL	e.g. Rent, Utilities, Supplier
amount	NUMERIC(12,2)	NOT NULL, CHECK (amount > 0)	Positive value only
currency	CHAR(3)	NOT NULL, DEFAULT 'MYR'	ISO 4217 currency code
date	DATE	NOT NULL	Transaction date
description	TEXT	NULLABLE	Free-text detail
source	VARCHAR(50)	NOT NULL	manual / pos_sync / csv_upload

sales_records — records all revenue events; separated from expenses to preserve 3NF and avoid mixed-sign amounts

Column	Type	Constraint	Notes
sale_id	UUID	PK, DEFAULT gen_random_uuid()	—

user_id	UUID	FK → users(user_id), NOT NULL	Tenant scoping
Category	VARCHAR(50)	Default NULL	Item category — may be null if not in POS export
amount	NUMERIC(12,2)	NOT NULL, CHECK (amount > 0)	Gross sale value
currency	CHAR(3)	NOT NULL, DEFAULT 'MYR'	ISO 4217 currency code
date	DATE	NOT NULL	Sale date
source	VARCHAR(50)	NOT NULL	pos_sync / manual / csv_upload
reference_id	VARCHAR(100)	NULLABLE	POS receipt or invoice reference

3.2.4 Module 4 — Staff

staff_members — master record of each staff member under a business

Column	Type	Constraint	Notes
staff_id	UUID	PK, DEFAULT gen_random_uuid()	—
user_id	UUID	FK → users(user_id), NOT NULL	Tenant scoping
name	VARCHAR(100)	NOT NULL	Staff full name
role	VARCHAR(80)	NOT NULL	e.g. Cashier, Chef, Supervisor
hourly_rate	NUMERIC(8,2)	NOT NULL, CHECK (hourly_rate > 0)	Used for labour cost calculation
employment_type	VARCHAR(20)	NOT NULL	full_time / part_time / contract
active	BOOLEAN	NOT NULL, DEFAULT TRUE	Soft delete flag

shifts — records each worked shift; feeds Google DeepMind labour cost and scheduling analysis

Column	Type	Constraint	Notes
shift_id	UUID	PK, DEFAULT gen_random_uuid()	—
staff_id	UUID	FK → staff_records(staff_id), NOT NULL	Staff member referenced
user_id	UUID	FK → users(user_id), NOT NULL	Tenant scoping for RLS
date	DATE	NOT NULL	Shift date
hours_worked	NUMERIC(4,2)	NOT NULL, CHECK (hours_worked > 0)	Actual hours logged
peak_flag	BOOLEAN	NOT NULL, DEFAULT FALSE	Marks high-traffic shift periods
labour_cost	NUMERIC(10,2)	GENERATED ALWAYS AS (hours_worked * hourly_rate)	Computed at DB level

3.2.5 Module 5 — AI & Briefing

briefings — stores each generated Google DeepMind briefing; serves the 24h cache layer

Column	Type	Constraint	Notes
briefing_id	UUID	PK, DEFAULT gen_random_uuid()	—
user_id	UUID	FK → users(user_id), NOT NULL	Tenant scoping
generated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Inference timestamp
briefing_date	DATE	NOT NULL	Business date the briefing covers
content_json	JSONB	NOT NULL	Parsed Google DeepMind output — action items, flags
cached_until	TIMESTAMPTZ	NOT NULL	TTL expiry; 24h from generated_at
status	VARCHAR(20)	NOT NULL, DEFAULT 'active'	active / expired / regenerating

chat_messages — Persistent conversation history between owner and Zara

Column	Type	Constraint	Notes
chat_id	UUID	PK, DEFAULT gen_random_uuid()	—
user_id	UUID	FK → users(user_id), NOT NULL	Tenant scoping
role	VARCHAR(50)	NOT NULL	user / assistant — CHECK constraint enforced
content	TEXT	NOT NULL	Full message text content
context_snapshot	TEXT	Default NULL	Optional: business context string used for this message
created_at	TIMESTAMPTZ	DEFAULT now()	Message timestamp — indexed with user_id for fast retrieval

business_rules — per-tenant configurable rules that influence Google DeepMind prompt construction

Column	Type	Constraint	Notes
rule_id	UUID	PK, DEFAULT gen_random_uuid()	—
user_id	UUID	FK → users(user_id), NOT NULL	Tenant scoping
module	VARCHAR(50)	NOT NULL	inventory / financial / staff / briefing
rule_key	VARCHAR(100)	NOT NULL	e.g. restock_lead_days, anomaly_threshold
rule_value	TEXT	NOT NULL	Stored as text; cast at application layer
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	—
		UNIQUE (user_id, module, rule_key)	Prevents duplicate rules per tenant

Summary

Every non-key column in each table depends solely on the primary key of that table; there are no transitive or partial dependencies. `sales_transactions` and `expenses` are kept as separate tables rather than a single ledger with a sign flag, because they carry structurally different attributes and mixing them would introduce nullable columns that violate 3NF. The `labour_cost` column in `shifts` is a generated column computed directly from `hours_worked` and `hourly_rate`, so it introduces no update anomaly. All M:N relationships — such as a staff member working many shifts, or an inventory item appearing in many transactions — are resolved through foreign key references into append-only child tables rather than repeated data in parent rows. Row-level security in Supabase enforces `user_id` scoping at the database layer across all tables, ensuring tenant isolation without application-level filtering.

4 Functional Requirements & Scope

4.1 Minimum Viable Product (MVP)

The following 5 core features will be built and demonstrated as indicated below:

#	Feature	Description
1	Business Health Dashboard	Centralized real-time view of financial KPIs, inventory status, staff utilization, and AI-generated daily briefing.
2	Ingredient & Inventory Manager	Tracks stock levels, predicts demand from historical data, flags expiry risks, and generates automated restocking recommendations.
3	Financial Monitor	Continuously monitors expenses, salaries, and subscriptions. Generates burn rate reports and profit forecasts with anomaly alerts.
4	AI Daily Briefing	Google Gemma 4 26B synthesizes daily operational data into a concise summary with prioritized action items for the SME owner.
5	Basic Rule Customization	Users define operational constraints (e.g., max weekly staff hours, reorder thresholds) that guide AI recommendation behavior.
6	AI Chatbot Assistant	A smart chatbot that turns business data into a conversation, answering queries like “What’s low in stock?” with real-time insights and actionable recommendations for instant decision-making.

4.2 Non-Functional Requirements (NFRs)

Below are the qualities that we ensured within the system, which are crucial for the reliability and scalability of the system:

Quality	Requirement	Implementation
Scalability	Platform must handle concurrent data ingestion and Google DeepMind calls during peak operating hours (e.g. lunch/dinner rush)	Stateless backend services; AWS ASG for horizontal scaling; ELB for traffic distribution
Reliability	Dashboard and inventory alerts must maintain 99% uptime; failed data syncs must not corrupt existing records	Idempotent data writes; transactional DB inserts; health-check endpoints per service
Maintainability	Each functional module (Financial, Inventory, Staff, AI) must be independently deployable	Modular service structure; independent CI/CD pipelines per module via GitHub Actions
Token Latency	Google DeepMind API calls must return responses within 3s under normal load (p95); briefing generation may use async with a 10s budget	5s timeout on sync calls; async queue for briefing generation; user-facing loading states
Cost Efficiency	Average Google DeepMind token usage per user session must stay within the defined budget threshold	Fixed-length prompt templates; input truncation/chunking before Google DeepMind injection; 24h Redis cache for repeated queries
Data Integrity	Inventory and financial records must reflect an accurate state at all times; no orphaned or stale entries	3NF-normalized schema; FK constraints enforced at DB level; soft-delete pattern for audit trails

4.3 Out of Scope / Future Enhancements

The following capabilities are planned as part of the long-term product vision but are excluded from this hackathon scope:

- Multi-user support with role-based access control (Owner, Manager, Staff)
- Native mobile application (iOS & Android)
- In-app messaging between staff and management
- External POS API integration (real-time webhook-based sync)
- Seasonal promotion engine, like discount codes, voucher generation, loyalty tracking
- Scheduled reporting, like automated weekly/monthly email digests
- Multi-branch support for SMEs operating across locations

5 Monitor, Evaluation, Assumptions & Dependencies

5.1 Technical Evaluation

5.1.1 Grayscale Rollout & A/B Testing

New AI recommendation features (e.g., updated Google DeepMind prompt strategies for inventory forecasting) will follow a phased rollout:

1. New feature released to 5% of active user sessions initially
2. System monitors key metrics: recommendation acceptance rate, latency, error rate
3. If no anomalies are detected over 48 hours, rollout expanded to 25% → 50% → 100%
4. A/B testing compares two Google DeepMind prompt variants to determine which produces higher-quality recommendations (measured by user acceptance rate and override frequency)

5.1.2 Emergency Rollback (ER) & Golden Release

Each production deployment is tagged as a versioned release. The most recent stable deployment is designated the Golden Release. If a P1 or P2 event is triggered:

- Automated CI/CD pipeline halts current deployment
- Golden Release is automatically redeployed within 5 minutes
- Traffic is rerouted via ELB to stable instances
- Incident is logged, and a post-mortem is scheduled within 24 hours

5.2 Priority Matrix

Priority	Trigger Condition	Condition Detail	Action
P1 – Critical	Google DeepMind API error rate > 5% or	Google DeepMind API error rate > 5% or dashboard unavailable > 2 min	Immediate emergency rollback to Golden

	dashboard unavailable > 2 min		Release; notify the on-call team
P2 – High	Inventory sync failure or financial data inconsistency detected	Inventory sync failure or financial data inconsistency detected	Trigger data reconciliation job; disable the affected module; alert the SME owner
P3 – Medium	AI briefing latency > 10s or recommendation quality degrades	AI briefing latency > 10s or recommendation quality degrades	Switch to cached fallback briefing; queue async regeneration
P4 – Low	Minor UI rendering issues or non-critical feature errors	Minor UI rendering issues or non-critical feature errors	Log issue; schedule fix in next sprint; no user impact expected

5.3 Monitoring Plan

The following monitoring strategy ensures system health is continuously observable:

- **Application Metrics:** Request latency, error rate, and Google DeepMind API response time are tracked via GCP Cloud Monitoring dashboards.
- **Database Health:** Supabase PostgreSQL connection pool utilisation, query execution time, and RLS policy evaluation are monitored via Supabase's built-in dashboard.
- **AI Quality Metrics:** Google DeepMind response schema validation pass rate and user override rate on recommendations are logged per session to Cloud Logging.
- **Cache Performance:** Google DeepMind Context Cache hit and miss ratios are tracked at the FastAPI service level and exported as custom metrics to Cloud Monitoring, enabling evaluation of caching effectiveness and token cost savings over time.
- **Alerting:** Cloud Monitoring alerting policies trigger notification channels like email, Slack webhook, or customized bot, for P1 and P2 conditions. P1 alerts fire immediately on health

check failure, P2 alerts fire after a sustained threshold breach of 3 minutes to suppress transient spikes.

5.4 Assumptions

The following operational and environmental conditions are assumed to be valid during development and deployment:

- SME owners have stable internet connectivity on the devices used to access the dashboard
- Business data (inventory, financial records) is available for at least the past 30 days at onboarding to enable meaningful AI pattern detection
- The Google Gemma 4 26B API remains available with consistent response times during the hackathon demonstration period
- POS system data can be exported in CSV format or entered manually into the platform
- Users will interact with the system in English; multi-language support is out of scope for MVP

5.5 External Dependencies

Tool / Service	Purpose	Risk & Mitigation
Google Gemma 4 26B API	Core LLM reasoning engine for prompt inference, pattern recognition, anomaly detection, and structured AI briefing generation	HIGH — API rate limits or downtime disables all AI features. Mitigation: graceful degradation to Google DeepMind Context Cache; user-facing fallback state displayed when live Google DeepMind is unavailable
GCP Cloud Run	Hosts all FastAPI backend microservices as independently	MEDIUM — Regional GCP outage affects availability.

	scalable, containerised Cloud Run services	Mitigation: Cloud Run health checks with automatic revision rollback; multi-service isolation means one service failure does not cascade
Supabase PostgreSQL	Persistent storage for all operational data — financial records, inventory transactions, staff data, and audit logs — with RLS-enforced tenant isolation	MEDIUM — Supabase outage affects all read/write operations. Mitigation: Supabase's managed high-availability PostgreSQL with automated failover; read-only fallback mode surfaced to the user during write unavailability
Supabase Storage	File storage for CSV uploads and receipt scans ingested from POS systems	LOW — Upload failures do not affect live dashboard data. Mitigation: client-side retry on upload failure; Edge Function idempotency prevents duplicate record insertion on retry
Google DeepMind Context Cache	In-memory cache at the FastAPI service layer reducing Google DeepMind token spend and response latency	LOW — Cache miss falls back transparently to a live Google DeepMind call. Mitigation: TTL-based invalidation set to 24 hours; cache warm-up on service start for high-frequency briefing queries
POS Integration	CSV or webhook data export from POS systems to populate inventory and financial modules	MEDIUM — Inconsistent POS export formats may cause import errors. Mitigation: schema validation and normalisation layer in Supabase Edge Functions before

		any records are committed to PostgreSQL
--	--	---

6 Project Management & Team Contributions

6.1 Project Timeline

The hackathon spans approximately 10 days, structured as a focused two-week sprint:

Period	Phase	Key Deliverables
Day 1–2	Project Planning & Architecture	Finalize SAD, define module boundaries, set up GitHub repo, configure AWS environment, establish DB schema draft
Day 3–4	Backend Core Development	Implement FastAPI service skeleton; PostgreSQL schema (3NF); data ingestion endpoints for inventory and financial modules
Day 5–6	AI Layer Integration	Google Gemma 4 26B Google DeepMind prompt design; context window construction; response parsing pipeline; Redis caching layer setup
Day 7–8	Frontend Dashboard Development	React.js dashboard: Financial Monitor, Inventory Manager, Staff overview; AI Daily Briefing display component
Day 9	Integration & Testing	End-to-end integration testing; NFR validation (latency, data integrity); bug fixes; monitoring setup
Day 10	Demo Preparation & Submission	Demo flow rehearsal; final SAD polish; deployment to production environment; submission package preparation

6.2 Team Member Roles

Member	Role	Responsibilities
Bryan Low Zhern Yang	Backend Lead	FastAPI microservices, PostgreSQL schema design (3NF), data ingestion pipeline, Google DeepMind API integration
Hor Yuan Li	AI / Prompt Engineer	Google Gemma 4 26B Google DeepMind prompt construction, context window management, response parsing, and caching strategy
Evin Kor Kar Hei	Frontend Lead	React.js dashboard, component design, API consumption, AI briefing display, user rule configuration UI
Kok Jia Yin	DevOps / QA Lead	AWS setup, CI/CD pipeline (GitHub Actions), monitoring plan, test cases, NFR validation
Chong Jinn Xiang	Product Owner/ System Analyst	Bridges the gap between SME business needs and technical implementation by translating operational inefficiencies into structured functional and non-functional requirements.

6.3 Recommendations

The following technical recommendations are proposed to enhance the platform's scalability, performance, and reliability beyond the MVP phase:

- **Redis Cache for Menu & Rule Data:** Cache business rules and inventory thresholds in Redis to reduce PostgreSQL query load and accelerate AI prompt construction.
- **Async Google DeepMind Processing with Job Queue:** Introduce a job queue (e.g. Celery + Redis) for non-time-critical Google DeepMind tasks (weekly summaries, menu advisor) to decouple AI calls from the request-response cycle.

- Prompt Versioning System: Maintain a versioned library of Google DeepMind prompt templates per feature to enable controlled experimentation and rollback of prompt strategies independently of code deployments.
- Read Replica for Supabase: Add a read replica for analytics and AI data queries, isolating heavy reads from transactional writes to maintain DB performance under load.
- Structured Logging with Correlation IDs: Tag all API requests, Google DeepMind calls, and DB queries with a shared correlation ID to enable end-to-end distributed tracing across the service stack.

Expected Outcome

The platform is expected to reduce manual operational monitoring effort for SME owners, enable proactive inventory and financial risk management, and establish a scalable AI-augmented foundation suitable for expansion into retail, service, and light manufacturing SME sectors.